PYTHON PROGRAMS

Data Structures and Algorithms

36 practical programs with clear titles and complete code

Topics include linked lists, stacks, queues, recursion, searching, trees, graphs, sorting and hashing.

Prepared as an exam-practice reference. Run each program separately in Python 3.

Contents

1. Student Record Management – Singly Linked List
2. Music Playlist Management – Doubly Linked List
3. Hospital Patient Management – Singly Linked List
4. Browser History Management – Doubly Linked List
5. Calculator Application – Postfix Stack Evaluation
6. Undo Operation in a Text Editor – Stack
7. Printer Job Scheduling – Linear Queue
8. Customer Service Management – Circular Queue
9. Expression Validation – Balanced Brackets
10. Postfix Expression Evaluation
11. Factorial – Recursion and Iteration
12. Fibonacci Series – Recursion and Iteration
13. Library Book Search – Binary Search
14. Examination Seating – Factorial
15. Rabbit Population – Fibonacci Series
16. Employee ID Search – Binary Search
17. Employee Record Management – Binary Search Tree
18. Student Mark Management – Binary Search Tree
19. Arithmetic Expression Evaluation – Expression Tree
20. Emergency Patient Scheduling – Min-Heap Priority Queue
21. GPS Route Navigation – Dijkstra’s Algorithm
22. Ambulance Route Optimization – Dijkstra’s Algorithm
23. Campus Network Installation – Prim’s Algorithm
24. Water Pipeline Distribution – Prim’s Algorithm
25. Electricity Distribution Network – Kruskal’s Algorithm
26. Road Construction Planning – Kruskal’s Algorithm
27. Student Rank List – Quick Sort
28. Online Shopping Prices – Quick Sort
29. Hospital Registration – Merge Sort
30. Employee Salary Processing – Merge Sort
31. Online Shopping Prices – Heap Sort
32. Student Rank List – Heap Sort Descending
33. Student Attendance – Linear and Binary Search
34. Inventory Product Search – Linear and Binary Search
35. Library Book Management – Hash Table with Chaining
36. Word Frequency Analysis – Hashing

Question 1: Student Record Management – Singly Linked List

```
class Node:
    def __init__(self, reg, name, marks):
        self.reg = reg
        self.name = name
        self.marks = marks
        self.next = None

head = None

def insert(reg, name, marks):
    global head
    new = Node(reg, name, marks)
    if head is None:
        head = new
    else:
        temp = head
        while temp.next:
            temp = temp.next
        temp.next = new

def search(reg):
    temp = head
    while temp:
        if temp.reg == reg:
            print(temp.name, temp.marks)
            return
        temp = temp.next
    print("Student not found")

def delete(reg):
    global head
    temp, prev = head, None
    while temp:
        if temp.reg == reg:
            if prev is None:
                head = temp.next
            else:
                prev.next = temp.next
            print("Student deleted")
            return
        prev, temp = temp, temp.next
    print("Student not found")

def display():
    temp = head
    while temp:
        print(temp.reg, temp.name, temp.marks)
        temp = temp.next

insert(101, "Arun", 85)
insert(102, "Bala", 90)
display()
search(102)
delete(101)
display()
```

Question 2: Music Playlist Management – Doubly Linked List

```
class Node:
    def __init__(self, song):
        self.song = song
        self.prev = None
        self.next = None

head = None

def insert(song):
    global head
    new = Node(song)
    if head is None:
        head = new
    else:
        temp = head
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp
```

```
def display_forward():
    temp = head
    while temp:
        print(temp.song, end=" ")
        temp = temp.next
    print()

def display_backward():
    temp = head
    if temp is None:
        return
    while temp.next:
        temp = temp.next
    while temp:
        print(temp.song, end=" ")
        temp = temp.prev
    print()

def search(song):
    temp = head
    while temp:
        if temp.song == song:
            print("Song found")
            return
        temp = temp.next
    print("Song not found")

def delete(song):
    global head
    temp = head
    while temp:
        if temp.song == song:
            if temp.prev:
                temp.prev.next = temp.next
            else:
                head = temp.next
            if temp.next:
                temp.next.prev = temp.prev
            print("Song deleted")
            return
        temp = temp.next
    print("Song not found")

insert("Song1")
insert("Song2")
insert("Song3")
print("Forward:")
display_forward()
print("Backward:")
display_backward()
search("Song2")
delete("Song2")
display_forward()
```

Question 3: Hospital Patient Management – Singly Linked List

```
class Node:
    def __init__(self, pid, name):
        self.pid = pid
        self.name = name
        self.next = None

head = None

def add(pid, name):
    global head
    new = Node(pid, name)
    if head is None:
        head = new
    else:
        temp = head
        while temp.next:
            temp = temp.next
        temp.next = new

def search(pid):
    temp = head
    while temp:
        if temp.pid == pid:
```

```
            print("Patient found:", temp.name)
            return
        temp = temp.next
    print("Patient not found")

def remove(pid):
    global head
    temp, prev = head, None
    while temp:
        if temp.pid == pid:
            if prev is None:
                head = temp.next
            else:
                prev.next = temp.next
            print("Patient removed")
            return
        prev, temp = temp, temp.next
    print("Patient not found")

def display():
    temp = head
    while temp:
        print(temp.pid, temp.name)
        temp = temp.next

add(1, "Arun")
add(2, "Bala")
display()
search(2)
remove(1)
display()
```

Question 4: Browser History Management – Doubly Linked List

```
class Node:
    def __init__(self, url):
        self.url = url
        self.prev = None
        self.next = None

head = None
current = None

def visit(url):
    global head, current
    new = Node(url)
    if head is None:
        head = new
    else:
        current.next = new
        new.prev = current
    current = new
    print("Visited:", url)

def backward():
    global current
    if current and current.prev:
        current = current.prev
        print("Current page:", current.url)
    else:
        print("No previous page")

def forward():
    global current
    if current and current.next:
        current = current.next
        print("Current page:", current.url)
    else:
        print("No next page")

def delete(url):
    global head, current
    temp = head
    while temp:
        if temp.url == url:
            if temp.prev:
                temp.prev.next = temp.next
            else:
                head = temp.next
```

```
            if temp.next:
                temp.next.prev = temp.prev
            if current == temp:
                current = temp.prev if temp.prev else temp.next
            print("History deleted")
            return
        temp = temp.next
    print("Website not found")

def display():
    temp = head
    while temp:
        print(temp.url)
        temp = temp.next

visit("Google")
visit("YouTube")
visit("Wikipedia")
backward()
forward()
delete("YouTube")
display()
```

Question 5: Calculator Application – Postfix Stack Evaluation

```
def evaluate(expression):
    stack = []
    for item in expression.split():
        if item.lstrip("-").isdigit():
            stack.append(int(item))
        else:
            b = stack.pop()
            a = stack.pop()
            if item == "+":
                stack.append(a + b)
            elif item == "-":
                stack.append(a - b)
            elif item == "*":
                stack.append(a * b)
            elif item == "/":
                stack.append(a / b)
            else:
                raise ValueError("Unsupported operator")
    if len(stack) != 1:
        raise ValueError("Invalid postfix expression")
    return stack.pop()

exp = input("Enter postfix expression: ")
print("Result:", evaluate(exp))
```

Question 6: Undo Operation in a Text Editor – Stack

```
stack = []

def perform(action):
    stack.append(action)
    print("Action performed:", action)

def undo():
    if stack:
        print("Undo:", stack.pop())
    else:
        print("Nothing to undo")

perform("Type Hello")
perform("Type Python")
perform("Delete text")
undo()
undo()
```

Question 7: Printer Job Scheduling – Linear Queue

```
from collections import deque

queue = deque()

def add_job(job):
```

```
    queue.append(job)
    print("Job added:", job)

def process_job():
    if queue:
        print("Printing:", queue.popleft())
    else:
        print("Queue is empty")

def display():
    print("Pending jobs:", list(queue))

add_job("Document1")
add_job("Document2")
add_job("Document3")
display()
process_job()
display()
```

Question 8: Customer Service Management – Circular Queue

```
size = 3
queue = [None] * size
front = rear = -1

def add(customer):
    global front, rear
    if (rear + 1) % size == front:
        print("Queue is full")
        return
    if front == -1:
        front = 0
    rear = (rear + 1) % size
    queue[rear] = customer
    print("Added:", customer)

def serve():
    global front, rear
    if front == -1:
        print("Queue is empty")
        return
    print("Serving:", queue[front])
    queue[front] = None
    if front == rear:
        front = rear = -1
    else:
        front = (front + 1) % size

def display():
    if front == -1:
        print("Queue is empty")
        return
    i = front
    while True:
        print(queue[i], end=" ")
        if i == rear:
            break
        i = (i + 1) % size
    print()

add("Arun")
add("Bala")
add("Cathy")
serve()
add("David")
display()
```

Question 9: Expression Validation – Balanced Brackets

```
def check(expression):
    stack = []
    pairs = {")": "(", "]": "[", "}": "{"}
    for ch in expression:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
```

```
    return len(stack) == 0

exp = input("Enter expression: ")
print("Balanced expression" if check(exp) else "Not balanced")
```

Question 10: Postfix Expression Evaluation

```
def evaluate(expression):
    stack = []
    for item in expression.split():
        if item.lstrip("-").isdigit():
            stack.append(int(item))
        else:
            b = stack.pop()
            a = stack.pop()
            if item == "+":
                stack.append(a + b)
            elif item == "-":
                stack.append(a - b)
            elif item == "*":
                stack.append(a * b)
            elif item == "/":
                stack.append(a / b)
    return stack.pop()

expression = "8 3 2 * + 5 -"
print("Result:", evaluate(expression))
```

Question 11: Factorial – Recursion and Iteration

```
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

def factorial_iterative(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact

n = int(input("Enter a non-negative number: "))
if n < 0:
    print("Factorial is not defined for negative integers")
else:
    print("Recursive:", factorial_recursive(n))
    print("Iterative:", factorial_iterative(n))
```

Question 12: Fibonacci Series – Recursion and Iteration

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = int(input("Enter number of terms: "))
if n < 0:
    print("Enter a non-negative number")
else:
    print("Recursive:")
    for i in range(n):
        print(fib(i), end=" ")
    print("\nIterative:")
    a, b = 0, 1
    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b
    print()
```

Question 13: Library Book Search – Binary Search

```
def binary_iterative(a, key):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
```

```
        if a[mid] == key:
            return mid
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

def binary_recursive(a, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if a[mid] == key:
        return mid
    elif a[mid] < key:
        return binary_recursive(a, mid + 1, high, key)
    return binary_recursive(a, low, mid - 1, key)

books = [10, 20, 30, 40, 50]
key = int(input("Enter book ID: "))
print("Iterative position:", binary_iterative(books, key))
print("Recursive position:",
      binary_recursive(books, 0, len(books) - 1, key))
```

Question 14: Examination Seating – Factorial

```
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

n = int(input("Enter number of students: "))
if n < 0:
    print("Enter a non-negative number")
else:
    print("Recursive:", factorial_recursive(n))
    print("Iterative:", factorial_iterative(n))
```

Question 15: Rabbit Population – Fibonacci Series

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = int(input("Enter number of terms: "))
if n < 0:
    print("Enter a non-negative number")
else:
    print("Using recursion:")
    for i in range(n):
        print(fib(i), end=" ")
    print("\nUsing iteration:")
    a, b = 0, 1
    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b
    print()
```

Question 16: Employee ID Search – Binary Search

```
def iterative_search(a, key):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] == key:
            return mid
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
```

```
    return -1

def recursive_search(a, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if a[mid] == key:
        return mid
    if a[mid] < key:
        return recursive_search(a, mid + 1, high, key)
    return recursive_search(a, low, mid - 1, key)

ids = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID: "))
print("Iterative:", iterative_search(ids, key))
print("Recursive:",
      recursive_search(ids, 0, len(ids) - 1, key))
```

Question 17: Employee Record Management – Binary Search Tree

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)

root = None
for x in [50, 30, 70, 20, 40]:
    root = insert(root, x)

print("Inorder:")
inorder(root)
print("\nPreorder:")
preorder(root)
print("\nPostorder:")
postorder(root)
print("\nSearch 40:", "Found" if search(root, 40) else "Not found")
```

Question 18: Student Mark Management – Binary Search Tree

```
class Node:
    def __init__(self, mark):
        self.mark = mark
        self.left = None
```

```
        self.right = None

def insert(root, mark):
    if root is None:
        return Node(mark)
    if mark < root.mark:
        root.left = insert(root.left, mark)
    elif mark > root.mark:
        root.right = insert(root.right, mark)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.mark, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.mark, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.mark, end=" ")

root = None
for mark in [80, 60, 90, 50, 70]:
    root = insert(root, mark)

print("Inorder:")
inorder(root)
print("\nPreorder:")
preorder(root)
print("\nPostorder:")
postorder(root)
```

Question 19: Arithmetic Expression Evaluation – Expression Tree

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def build_tree(expression):
    stack = []
    for item in expression.split():
        node = Node(item)
        if item in "+-*/":
            node.right = stack.pop()
            node.left = stack.pop()
        stack.append(node)
    if len(stack) != 1:
        raise ValueError("Invalid postfix expression")
    return stack.pop()

def evaluate(root):
    if root.value not in "+-*/":
        return int(root.value)
    a = evaluate(root.left)
    b = evaluate(root.right)
    if root.value == "+":
        return a + b
    if root.value == "-":
        return a - b
    if root.value == "*":
        return a * b
    if root.value == "/":
        return a / b

root = build_tree("8 2 + 3 *")
print("Result:", evaluate(root))
```

Question 20: Emergency Patient Scheduling – Min-Heap Priority Queue

```
import heapq

patients = []
heapq.heappush(patients, (2, "Arun"))
heapq.heappush(patients, (1, "Bala"))
heapq.heappush(patients, (3, "Cathy"))

print("Waiting patients:", sorted(patients))
priority, name = heapq.heappop(patients)
print("Treat patient:", name)
print("Remaining:", sorted(patients))
```

Question 21: GPS Route Navigation – Dijkstra's Algorithm

```
import heapq

graph = {
    'A': {'B': 4, 'C': 2},
    'B': {'A': 4, 'C': 1, 'D': 5},
    'C': {'A': 2, 'B': 1, 'D': 8, 'E': 10},
    'D': {'B': 5, 'C': 8, 'E': 2, 'F': 6},
    'E': {'C': 10, 'D': 2, 'F': 3},
    'F': {'D': 6, 'E': 3}
}

dist = {v: float('inf') for v in graph}
dist['A'] = 0
pq = [(0, 'A')]

while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
    for v, w in graph[u].items():
        nd = d + w
        if nd < dist[v]:
            dist[v] = nd
            heapq.heappush(pq, (nd, v))

for v in dist:
    print(v, ":", dist[v])
```

Question 22: Ambulance Route Optimization – Dijkstra's Algorithm

```
import heapq

graph = {
    'A': {'B': 10, 'C': 3},
    'B': {'A': 10, 'C': 1, 'D': 2},
    'C': {'A': 3, 'B': 1, 'D': 8, 'E': 2},
    'D': {'B': 2, 'C': 8, 'E': 7},
    'E': {'C': 2, 'D': 7}
}

dist = {v: float('inf') for v in graph}
parent = {v: None for v in graph}
dist['A'] = 0
pq = [(0, 'A')]

while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
    for v, w in graph[u].items():
        nd = d + w
        if nd < dist[v]:
            dist[v] = nd
            parent[v] = u
            heapq.heappush(pq, (nd, v))

path = []
v = 'E'
while v is not None:
    path.append(v)
    v = parent[v]
path.reverse()

print("Shortest route:", " -> ".join(path))
```

```
print("Minimum distance:", dist['E'])
```

Question 23: Campus Network Installation – Prim's Algorithm

```
import heapq

graph = {
    'A': {'B': 6, 'C': 3},
    'B': {'A': 6, 'C': 2, 'D': 5},
    'C': {'A': 3, 'B': 2, 'D': 3, 'E': 4},
    'D': {'B': 5, 'C': 3, 'E': 2},
    'E': {'C': 4, 'D': 2}
}

visited = {'A'}
pq = []
for v, w in graph['A'].items():
    heapq.heappush(pq, (w, 'A', v))

total = 0
print("Selected edges:")
while pq and len(visited) < len(graph):
    w, u, v = heapq.heappop(pq)
    if v in visited:
        continue
    visited.add(v)
    total += w
    print(u, "-", v, ":", w)
    for nxt, cost in graph[v].items():
        if nxt not in visited:
            heapq.heappush(pq, (cost, v, nxt))

print("Minimum total cost:", total)
```

Question 24: Water Pipeline Distribution – Prim's Algorithm

```
import heapq

graph = {
    'A': {'B': 7, 'C': 5},
    'B': {'A': 7, 'C': 8, 'D': 7},
    'C': {'A': 5, 'B': 8, 'D': 6, 'E': 5},
    'D': {'B': 7, 'C': 6, 'E': 4, 'F': 3},
    'E': {'C': 5, 'D': 4, 'F': 2},
    'F': {'D': 3, 'E': 2}
}

visited = {'A'}
pq = []
for v, w in graph['A'].items():
    heapq.heappush(pq, (w, 'A', v))

total = 0
while pq and len(visited) < len(graph):
    w, u, v = heapq.heappop(pq)
    if v in visited:
        continue
    visited.add(v)
    total += w
    print(u, "-", v, ":", w)
    for nxt, cost in graph[v].items():
        if nxt not in visited:
            heapq.heappush(pq, (cost, v, nxt))

print("Minimum total cost:", total)
```

Question 25: Electricity Distribution Network – Kruskal's Algorithm

```
edges = [
    (4, 'A', 'B'), (2, 'A', 'C'), (5, 'B', 'C'),
    (10, 'B', 'D'), (3, 'C', 'D'), (7, 'C', 'E'),
    (1, 'D', 'E')
]

parent = {x: x for x in 'ABCDE'}

def find(x):
```

```
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

total = 0
for w, u, v in sorted(edges):
    a, b = find(u), find(v)
    if a != b:
        parent[a] = b
        total += w
        print(u, "-", v, ":", w)

print("Minimum total cost:", total)
```

Question 26: Road Construction Planning – Kruskal's Algorithm

```
edges = [
    (8, 'A', 'B'), (4, 'A', 'C'), (3, 'B', 'C'),
    (6, 'B', 'D'), (5, 'C', 'D'), (7, 'C', 'E'),
    (2, 'D', 'E'), (9, 'D', 'F'), (4, 'E', 'F')
]

parent = {x: x for x in 'ABCDEF'}

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

total = 0
for w, u, v in sorted(edges):
    a, b = find(u), find(v)
    if a != b:
        parent[a] = b
        total += w
        print(u, "-", v, ":", w)

print("Minimum total cost:", total)
```

Question 27: Student Rank List – Quick Sort

```
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[0]
    left = [x for x in a[1:] if x <= pivot]
    right = [x for x in a[1:] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

marks = [75, 90, 65, 80, 70]
print("Sorted marks:", quick_sort(marks))
```

Question 28: Online Shopping Prices – Quick Sort

```
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[0]
    left = [x for x in a[1:] if x <= pivot]
    right = [x for x in a[1:] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

prices = [500, 200, 800, 300, 100]
print("Sorted prices:", quick_sort(prices))
```

Question 29: Hospital Registration – Merge Sort

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = j = 0
```

```
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]

ids = [104, 101, 105, 102, 103]
print("Sorted IDs:", merge_sort(ids))
```

Question 30: Employee Salary Processing – Merge Sort

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]

salary = [30000, 20000, 50000, 25000]
print("Sorted salaries:", merge_sort(salary))
```

Question 31: Online Shopping Prices – Heap Sort

```
def heap_sort(a):
    n = len(a)

    def heapify(n, i):
        largest = i
        left, right = 2 * i + 1, 2 * i + 2
        if left < n and a[left] > a[largest]:
            largest = left
        if right < n and a[right] > a[largest]:
            largest = right
        if largest != i:
            a[i], a[largest] = a[largest], a[i]
            heapify(n, largest)

    for i in range(n // 2 - 1, -1, -1):
        heapify(n, i)

    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(i, 0)
    return a

prices = [500, 100, 400, 200, 300]
print("Sorted prices:", heap_sort(prices))
```

Question 32: Student Rank List – Heap Sort Descending

```
def heap_sort(a):
    n = len(a)

    def heapify(n, i):
        largest = i
        l, r = 2 * i + 1, 2 * i + 2
        if l < n and a[l] > a[largest]:
            largest = l
        if r < n and a[r] > a[largest]:
            largest = r
        if largest != i:
            a[i], a[largest] = a[largest], a[i]
            heapify(n, largest)
```

```
    for i in range(n // 2 - 1, -1, -1):
        heapify(n, i)

    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(i, 0)
    return a

marks = [75, 95, 60, 85, 70]
print("Descending order:", heap_sort(marks)[::-1])
```

Question 33: Student Attendance – Linear and Binary Search

```
def linear_search(a, key):
    count = 0
    for i, value in enumerate(a):
        count += 1
        if value == key:
            return i, count
    return -1, count

def binary_search(a, key):
    low, high = 0, len(a) - 1
    count = 0
    while low <= high:
        mid = (low + high) // 2
        count += 1
        if a[mid] == key:
            return mid, count
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, count

a = [101, 102, 103, 104, 105, 106, 107]
key = 106
p1, c1 = linear_search(a, key)
p2, c2 = binary_search(a, key)
print("Linear position:", p1)
print("Linear comparisons:", c1)
print("Binary position:", p2)
print("Binary comparisons:", c2)
```

Question 34: Inventory Product Search – Linear and Binary Search

```
def linear_search(a, key):
    count = 0
    for i, value in enumerate(a):
        count += 1
        if value == key:
            return i, count
    return -1, count

def binary_search(a, key):
    low, high = 0, len(a) - 1
    count = 0
    while low <= high:
        mid = (low + high) // 2
        count += 1
        if a[mid] == key:
            return mid, count
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, count

codes = [10, 20, 30, 40, 50, 60, 70]
key = 40
p1, c1 = linear_search(codes, key)
p2, c2 = binary_search(codes, key)
print("Linear search:", p1, "Comparisons:", c1)
print("Binary search:", p2, "Comparisons:", c2)
```

Question 35: Library Book Management – Hash Table with Chaining

```
size = 5
table = [[] for _ in range(size)]

def insert(book_id):
    index = book_id % size
    table[index].append(book_id)
    print("Book inserted:", book_id)

def search(book_id):
    index = book_id % size
    print("Book found" if book_id in table[index] else "Book not found")

def delete(book_id):
    index = book_id % size
    if book_id in table[index]:
        table[index].remove(book_id)
        print("Book deleted")
    else:
        print("Book not found")

insert(101)
insert(106)
insert(102)
print("Hash table:", table)
search(106)
delete(106)
print("Hash table:", table)
```

Question 36: Word Frequency Analysis – Hashing

```
article = input("Enter an article: ")
words = article.lower().split()
frequency = {}

for word in words:
    word = word.strip(".,!?;:")
    if word:
        frequency[word] = frequency.get(word, 0) + 1

print("Word frequencies:")
for word, count in frequency.items():
    print(word, ":", count)

if frequency:
    maximum = max(frequency.values())
    common = [word for word, count in frequency.items()
              if count == maximum]
    print("Most frequent word(s):", common)
```